

MODULE 21 · PART 7 - PROOF AND PERFORMANCE

Reading a Plan, Making it Fast

Which numbers matter, which changes help, and which are folklore.

PART 7 of 8 MODULE 21 of 26 RUNS about 32 min

Why this comes after module twenty

- ◆ **A fast wrong answer is worse than a slow one**, because it arrives in time to be acted on.
- ◆ **And most optimisation changes what a query computes**, not merely how. Aggregating earlier is a different query.
- ◆ **So the assertions from module twenty are what let you optimise safely**, by telling you when a rewrite changed the answer.
- ◆ **Optimise a query with no assertions and you are guessing** that the faster version returns the same thing.

THE TAKEAWAY

Every measured improvement in this module is a rewrite. Module eighteen's EXCEPT in both directions is how you check a rewrite returned the same rows.

Almost every optimisation in this course has been the same one: **do less work**, not the same work faster.

What a plan actually is

- ◆ **A tree of operators**, each taking rows from its children and producing rows for its parent.
- ◆ **Read it from the bottom up**, because that is the direction the rows travel. The top is the last thing to happen.
- ◆ **Every operator has two numbers worth reading**: how many rows came out, and how long it took.
- ◆ **And most of the rest is noise**, at least until you have found the operator holding the time.

THE TAKEAWAY

The tree is the query's actual execution order, which is neither the order you wrote nor module one's logical order. It is a third thing.

Reading a plan out loud

WHAT YOU SEE	WHAT IT MEANS
SEQ_SCAN 7,069,736 rows	every row was read; no filter got pushed down here
FILTER 1,132,784 rows	84% of them were discarded, after being read
HASH_JOIN build side 412	the small side fits in memory; this join is cheap
ORDER_BY / SORT 7,069,736	the expensive one, and the row count is the reason
rows=1 ... actual rows=4,180,211	the planner chose for a query you did not run

THE TAKEAWAY

Five lines, and the fourth is the one to look for first. In analytical SQL the sort is the default suspect, and it is usually right.

EXPLAIN and EXPLAIN ANALYZE

```
1 EXPLAIN          <query>  -- the plan, with estimates. Does not run.
2 EXPLAIN ANALYZE  <query>  -- runs it, and reports actual rows and time.
3
4 -- PostgreSQL:  EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
5 -- DuckDB:      EXPLAIN ANALYZE          (actuals only, no estimates)
6 -- Redshift:    EXPLAIN, then STL_/SYS_ views for what actually happened
7 -- Snowflake:   the query profile in the UI, or GET_QUERY_OPERATOR_STATS
8 -- SQL Server:  SET STATISTICS PROFILE ON, or the graphical plan
```

2 ANALYZE executes the query. On a destructive statement, wrap it in a transaction you roll back.

THE TAKEAWAY

EXPLAIN is free and tells you what the planner believes. EXPLAIN ANALYZE costs a full execution and tells you what happened. You almost always want the second.

Which to use when

- ◆ **EXPLAIN** when the query is too slow to finish, which is the case where you need it most and cannot run it.
- ◆ **EXPLAIN ANALYZE** for everything else, because estimates alone cannot reveal that the estimates were wrong.
- ◆ **Never tune from EXPLAIN alone on an unfamiliar query**, since the numbers you are reading are the planner's opinion.
- ◆ **And on a query that will not finish, EXPLAIN is still informative**, because a nested loop over a large estimate is visible before you run it.

THE TAKEAWAY

The awkward case is a query too slow to ANALYZE. Run it against a sample, or a shorter date range, and read the plan shape rather than the timings.

A real plan, with the noise removed

```
1 SELECT s.region_key, SUM(f.net_amount)
2   FROM fact_sales_line f JOIN dim_store s USING (store_key)
3  WHERE f.sold_ts >= DATE '2026-06-01'
4  GROUP BY s.region_key;
5
6  operator                rows out
7  -----
8  PERFECT_HASH_GROUP_BY      7    <- last
9  PROJECTION                1,132,784
10 HASH_JOIN                  1,132,784
11 SEQ_SCAN fact_sales_line 7,069,736    <- first
```

- 12 Seven million rows in at the bottom, seven rows out at the top. The shape of every analytical query.
- 11 The filter reduced it to 1,132,784, which is 16%. That reduction is the single most important number here.

THE TAKEAWAY

Taken from an EXPLAIN ANALYZE on the capstone database, with the box-drawing stripped. Four operators, and the interesting one is the reduction between the bottom two.

The same operator, five names

WHAT IT DOES	WHAT IT IS CALLED
Read a whole table	Seq Scan, SEQ_SCAN, Table Scan, Full Scan
Join by hashing the small side	Hash Join, HASH_JOIN, Hash Match
Join by scanning one side per row	Nested Loop, NESTED_LOOP, Loop Join
Arrange rows	Sort, ORDER_BY, Sort (Top N)
Group	HashAggregate, HASH_GROUP_BY, Stream Aggregate

THE TAKEAWAY

Five vocabularies for one set of ideas. The names differ; the three expensive things do not, and neither does the method for finding them.

The two numbers worth reading

- ◆ **Rows out of each operator.** Where does the row count fall, and how far? That is where the work stops being expensive.
- ◆ **Time held by each operator.** Not cumulative time, the time this one operator spent. One usually holds most of it.
- ◆ **Not the cost estimate**, which is a unitless number for comparing plans and means nothing on its own.
- ◆ **Find the operator holding the time, then look at its inputs.** That is the whole method.

THE TAKEAWAY

People read plans looking for something that looks wrong. The method is narrower: find where the time is, then ask why that operator received so many rows.

Estimated rows against actual rows. When those diverge, the planner chose a strategy for a query you did not run.

What a misestimate looks like

```
1 -- PostgreSQL's format, where both numbers are shown
2
3 Nested Loop (cost=0.43..8.9 rows=1 width=16)
4           (actual time=0.02..41822.11 rows=4,180,211 loops=1)
5
6 -- estimated: 1 row.  actual: 4.18 million.
7 -- a nested loop is an excellent plan for 1 row
8 -- and a catastrophe for 4 million.
```

4 rows=1 against actual rows=4,180,211. A factor of four million, and the plan is not wrong for what it expected.

THE TAKEAWAY

PostgreSQL shows both on every line, which is why its plans are the ones worth learning to read. DuckDB's EXPLAIN ANALYZE shows actuals only.

Why estimates go wrong

- ◆ **Stale statistics.** The table grew tenfold and nobody analysed it. The commonest cause and the easiest fix.
- ◆ **Correlated predicates.** The planner multiplies selectivities as if independent, and city and postcode are not independent.
- ◆ **Expressions it cannot see through.** A function on a column, a JSON extraction, a cast. No statistics exist for those.
- ◆ **And anything behind a CTE fence**, on an engine that materialises, because the estimate stops at the boundary.

THE TAKEAWAY

The third one is module seventeen's JSON column arriving here. A predicate on an extracted key has no statistics, so the planner guesses, and its guess is a constant.

A nightly aggregate has slowed from four minutes to fifty. The plan shows a nested loop join where it previously showed a hash join. Nothing in the query has changed.

The plan line reads: estimated 1 row, actual 4,180,211. The dimension table it joins to was reloaded last week and is now 40 times larger.

What is the right first action?

- A Restructure the query so the planner cannot choose a nested loop, since that operator is the one holding the time.
- B Increase the memory available to the query, so the nested loop has more room to work in.
- C Add an index on the join column, which will make the nested loop's inner lookups cheap.
- D Update the statistics on the reloaded table, because the planner chose a good plan for a table 40 times smaller.

◆ PAUSE AND DECIDE

What is the right first action?

A Restructure the query

Fights the planner rather than informing it. It would work, and every future query against that table has the same wrong estimate.

C Add an index

Genuinely makes the nested loop faster, and leaves the planner believing the table has one row. It fixes this query only.

B More memory

A nested loop over four million rows is not memory-bound. It is doing four million lookups, and each one is cheap.

D Update the statistics

The estimate is the defect. One command, and every query against that table gets a plan chosen for the table that exists.

THE SIGNAL

"Nothing in the query has changed." A query that slows down without changing has had its inputs change, and the planner is the first place that shows.

In analytical SQL there are really **three expensive things**. Everything else is a scan.

A spill is not a slowdown. It is **a step change**, and it happens without warning at a threshold.

Spills, and what to do about them

- ◆ **The symptom is a cliff, not a slope.** A small data increase producing an enormous time increase is the signature.
- ◆ **Look for spill or temp bytes in the plan,** which most engines report. On Postgres it is 'external merge Disk:'.
- ◆ **And the real fix is fewer rows,** again, because a sort that fits does not spill and one that fits easily never will.
- ◆ **More memory is the legitimate second answer here,** because a spill is genuinely a memory problem rather than a shape problem.

THE TAKEAWAY

This is the one case where more resources is a defensible first response, and even here, halving the rows entering the sort is usually cheaper than doubling the memory.

Three expensive things

OPERATOR	WHEN IT HURTS
Sort	Window functions, ORDER BY, merge joins. Cost grows faster than linearly.
Hash build	Joins and GROUP BY. Cheap until the build side stops fitting in memory.
Spill to disk	Either of the above, too large. The cliff, where time jumps by an order of magnitude.
Scan	Rarely. Reading is fast; it is what you do with the rows afterwards that is not.

THE TAKEAWAY

The sort is the one this course keeps meeting, because every window function needs one and most analytical queries have several.

Predicate pushdown, and what blocks it

```
1 -- filter inside a CTE, used outside it
2 WITH c AS (SELECT * FROM fact_sales_line)
3 SELECT COUNT(*) FROM c WHERE store_key = 41;      0.086s
4
5 -- the same filter, written inline
6 SELECT COUNT(*) FROM fact_sales_line
7 WHERE store_key = 41;      0.000s
8
9 -- measured. The filter did not reach the scan in the first form.
```

7 Unmeasurable, because the filter reached the scan and the engine skipped whole row groups.

THE TAKEAWAY

Measured on the capstone database. Pushdown is the optimisation you get for free when nothing blocks it, and a block costs everything it would have saved.

What blocks a pushdown

- ◆ **A materialisation fence.** MATERIALIZED on a CTE, or an engine that materialises every CTE, as module ten covered.
- ◆ **A function wrapping the column.** WHERE UPPER(name) = 'X' cannot use an index or a zone map on name.
- ◆ **A window function in between,** because the engine cannot know a filter above it is safe to apply below it.
- ◆ **And filtering in the wrong clause,** which is module one: a WHERE runs before windows and a QUALIFY after.

THE TAKEAWAY

The second one is the most common by far. A cast or a function on the filtered column turns an indexed lookup into a full scan, silently.

Every window function needs **a sort**. Four different window specifications means four of them.

Measured, on 7.07 million rows

```
1 -- three aggregates, one shared window
2 SUM(net_amount) OVER w, AVG(net_amount) OVER w, COUNT(*) OVER w
3 WINDOW w AS (PARTITION BY store_key ORDER BY sold_ts)
4                                     4.69s
5
6 -- three aggregates, three different windows
7 SUM(...) OVER (PARTITION BY store_key ORDER BY sold_ts),
8 AVG(...) OVER (PARTITION BY store_key ORDER BY date_key),
9 COUNT(*) OVER (PARTITION BY product_key ORDER BY sold_ts)
10                                  7.16s
```

10 Half as much again, for the same three aggregates over the same rows. The difference is entirely the extra sorts.

THE TAKEAWAY

Measured on the capstone database. 4.69 seconds against 7.16, and the only difference is whether the three windows agree on how the rows should be arranged.

How to share a sort

- ◆ **Use a named WINDOW clause**, so the specification is written once and every function referencing it provably matches.
- ◆ **Specifications must match exactly to share.** Same partition, same ordering, and usually the same frame.
- ◆ **Frames can differ and still share the sort** on most engines, because the frame is applied after the arrangement.
- ◆ **And a different ORDER BY is a different sort**, even if the partition matches. That is the one people get wrong.

THE TAKEAWAY

Writing the same specification out four times works and is fragile. Somebody edits one of them, the sharing silently stops, and the query gets slower for no visible reason.

A store performance query computes six window functions over seven million sales lines and takes eleven minutes. Four of them use `PARTITION BY store_key ORDER BY sold_ts`; two use the same partition with `ORDER BY date_key`.

The team has added an index on `sold_ts` and doubled the cluster. Neither changed the runtime materially. The plan shows six sort operators.

What should they do?

- A Add a second index on `date_key`, so that both orderings are served by an index rather than a sort.
- B Six specifications means six sorts. Share one where the logic allows, and accept a second only where it genuinely differs.
- C Increase the memory per query further, since six sorts over seven million rows will be spilling.
- D Split the query into six queries, one per window function, and join the results back together.

◆ PAUSE AND DECIDE

What should they do?

A A second index

An index can serve a sort of a base table, and these sorts are of an intermediate result. There is nothing for the index to help.

C More memory

Stops the spilling and leaves six sorts. Worth doing if they are spilling, and it is treating the smaller half.

B Share the specifications

Six sorts become two. Measured on the same data, sharing three windows instead of splitting them was 1.5 times faster.

D Six separate queries

Six sorts become six sorts plus five joins. It also reads the fact table six times instead of once.

THE SIGNAL

"The plan shows six sort operators." The plan is telling them the answer directly. An index and a bigger cluster both address the cost of a sort rather than the number of them.

Reduce to the question's grain **before** the expensive operator, not after.

Measured, again on the capstone database

```
1 -- window over every sales line
2 ROW_NUMBER() OVER (PARTITION BY store_key ORDER BY sold_ts)
3 FROM fact_sales_line          7,069,736 rows      1.36s
4
5 -- reduce to store-days first, then window
6 ROW_NUMBER() OVER (PARTITION BY store_key ORDER BY d)
7 FROM (SELECT DISTINCT store_key, sold_ts::DATE d
8       FROM fact_sales_line)    237,312 rows      0.20s
```

8 Nearly seven times faster, and the deduplication is not free. The sort it avoids costs more than the deduplication does.

THE TAKEAWAY

Thirty times fewer rows into the sort. No index, no configuration, no cluster change. The query simply stopped sorting rows it was going to collapse anyway.

Where to put the reduction

- ◆ **Before the sort**, which is the biggest single win and the one this course has made four times.
- ◆ **Before the join**, because a fact aggregated to the join's grain fans out less and hashes smaller.
- ◆ **Not after either**. Filtering the output of an expensive operator means you paid for rows you discarded.
- ◆ **And the engine often does this for you**, by pushing predicates down. Often, not always, and never through a fence.

THE TAKEAWAY

The predicates an engine can push down are the ones it can see. A filter on the far side of a materialised CTE, or on a function's output, stays where you wrote it.

Most performance advice is **repeated rather than measured**. So I measured it.

The slow operator is **where the problem shows**. Its input is **what the problem is**.

The cache trap

```
1 SELECT store_key, SUM(net_amount)
2   FROM fact_sales_line GROUP BY store_key;
3
4  run 1      4.827s    <- cold
5  run 2      0.053s
6  run 3      0.052s    <- warm
7
8  -- 91x faster, and nothing was changed between the runs.
```

8 You can 'improve' any query by ninety times by running it twice and reporting the second number.

THE TAKEAWAY

Measured on the capstone database. This is the single easiest way to convince yourself and a reviewer that an optimisation worked when it did nothing at all.

Benchmarking a change honestly

- ◆ **Warm both versions first**, then measure. Or clear the cache for both. Never one of each.
- ◆ **Run each several times and take the median**, because a single run measures whatever else the machine was doing.
- ◆ **Alternate them**, rather than running all of A then all of B, so that drift affects both equally.
- ◆ **And never compare a cold run of the old against a warm run of the new**, which is what happens by default.

THE TAKEAWAY

The default measurement procedure is: run the old one, change it, run the new one. That procedure produces a speedup from the cache regardless of what you changed.

And measure the right thing

- ◆ **Wall clock on a shared warehouse is noise**, because it includes queueing and whatever else was running.
- ◆ **Rows scanned and bytes scanned are stable**, and on a per-query-billed engine they are what you actually pay.
- ◆ **So report the row count into the expensive operator**, which is the thing you changed and does not vary with load.
- ◆ **Seven million to 237,000 is a claim anybody can reproduce.** 'It got faster' is not.

THE TAKEAWAY

A change that reduces rows scanned by 30 times is an improvement whatever the clock says. A change that only moved the clock might be the cache.

Folklore, measured

```
1 COUNT(*)           0.000s      7,069,736
2 COUNT(1)           0.000s      7,069,736
3 COUNT(sale_key)     0.000s      7,069,736
4 COUNT(promo_key)    0.042s      2,686,679  <- different answer
5
6 SUM(net_amount) direct      0.020s
7 SUM(net_amount) via SELECT * 0.019s
```

- 4 The only one that differs, and it differs because it is a different question. COUNT of a nullable column skips nulls.
- 7 Identical, because the optimiser pruned the projection. SELECT * is not slow when the engine can see through it.

THE TAKEAWAY

Measured on the capstone database. Count of one is not faster than count of star; they compile to the same thing. Count of a nullable column is a different question entirely.

What the folklore gets wrong

- ◆ **COUNT(1) is not faster than COUNT(*).** They compile to the same operator, and both are metadata on a columnar store.
- ◆ **SELECT * is not slow in itself,** because the optimiser prunes unused columns. It is slow when nothing can prune it.
- ◆ **SELECT * is still bad,** for module eight's reason: the shape of your output changes when somebody adds a column.
- ◆ **Which is the pattern.** Most of the folklore is good advice with a wrong justification attached.

THE TAKEAWAY

SELECT * deserves to be avoided and not for performance. Giving a good rule a false reason means the rule gets dropped the moment somebody measures it.

A new SQL standards document mandates `COUNT(1)` over `COUNT(*)`, forbids `SELECT *` on performance grounds, requires tables to be joined smallest-first, and recommends replacing window functions with self-joins where possible.

An engineer is asked to review it before it goes to all teams.

What should the review say?

- A Approve. Each item is widely recommended, and a consistent standard is worth more than debating individual items.
- B Approve the `SELECT *` rule and drop the rest, since only that one has a measurable performance effect.
- C Three of the four are folklore. Keep the `SELECT *` rule for its real reason, which is output stability, not speed.
- D Reject it entirely. Performance standards cannot be written in advance and must be decided per query from its plan.

◆ PAUSE AND DECIDE

What should the review say?

A Approve as written

Consistency around three false claims teaches a whole organisation to optimise the wrong things, and the fourth item makes queries slower.

C Three are folklore

COUNT(1) is identical, join order is the planner's job, and self-joins are slower than the windows they replace.

B Keep SELECT * only

*Right conclusion, wrong reason. SELECT * measured identically here; keeping it for speed means dropping it when somebody measures.*

D Reject entirely

Overcorrects. 'Reduce to the question's grain before the sort' is a standard worth writing, and it applies to every query.

THE SIGNAL

"Replacing window functions with self-joins." That item is not merely useless, it reverses the advice: a self-join re-reads the table where a window makes one pass.

What actually helps

- ◆ **Fewer rows into the expensive operator.** Measured at nearly seven times on this data, and it needs no configuration.
- ◆ **Fewer sorts,** by sharing window specifications. Measured at one and a half times.
- ◆ **Correct statistics,** so the planner chooses for the query you are running rather than one you are not.
- ◆ **And materialising a result several things read,** which is module ten, and which turns N computations into one.

THE TAKEAWAY

Four changes. Three of them are things you write, and one is a maintenance command. None of them is a configuration setting or a bigger machine.

The planner reorders joins better than you do. **Until it does not**, and then it is the statistics.

When to override the planner

- ◆ **Almost never, and later than you think.** The planner has information you do not, including memory available right now.
- ◆ **After you have checked the estimates,** because a wrong estimate is the cause in most cases where the plan looks silly.
- ◆ **And a hint is a decision frozen at the moment you wrote it,** which will be wrong when the data changes and will not tell you.
- ◆ **Most engines make this hard deliberately.** Postgres has no join hints at all, which is a design position rather than an oversight.

THE TAKEAWAY

A hint is the opposite of a statistic. One tells the planner what is true and lets it decide; the other tells it what to do and stops it deciding.

Partition pruning

```
1 -- prunes: the engine can skip partitions before reading
2 WHERE sold_date >= DATE '2026-06-01'
3
4 -- does not prune: a function wraps the partition column
5 WHERE YEAR(sold_date) = 2026
6
7 -- does not prune: the value is not knowable until the row is read
8 WHERE sold_date >= (SELECT MAX(d) FROM other_table) -- sometimes
9
10 -- the fix is always the same: compare the bare column to a constant
```

5 The commonest pruning failure there is. Rewrite it as a range on the bare column and the whole scan disappears.

THE TAKEAWAY

Pruning is the largest single lever on a partitioned or clustered table, and a function on the partition column silently turns it off.

Pruning, clustering and sort keys

- ◆ **Different names, one idea.** The engine keeps min and max per block and skips blocks that cannot match.
- ◆ **It works when the data is ordered by that column**, which is why a sort key or cluster key exists at all.
- ◆ **And it does nothing for a column the data is not ordered by**, where every block contains every value.
- ◆ **Which is why you get roughly one good pruning column**, and it is almost always the date.

THE TAKEAWAY

Zone maps on Redshift, clustering on Snowflake and BigQuery, row-group statistics in Parquet. The same mechanism, and it depends entirely on physical ordering.

A bigger machine is **the only fix that works on every query**, which is exactly why it is the wrong first answer.

A pipeline step takes 40 minutes and is on the critical path. The plan shows a scan at 12 seconds, a hash join at 30 seconds, and a sort holding 38 of the 40 minutes. The sort receives 7 million rows.

The proposal on the table is to double the cluster size, at a stated annual cost. The query computes a single ranking, and the report it feeds is at store-day grain.

What should be tried first?

- A Aggregate to store-day grain before the ranking, so the sort receives 237,000 rows instead of 7 million.
- B Double the cluster. The sort is genuinely the bottleneck, and more parallelism is the direct remedy for it.
- C Add an index on the ranking's ordering column, so the sort can be satisfied by an ordered read instead.
- D Move the step off the critical path by running it earlier, since the runtime itself cannot be reduced.

◆ PAUSE AND DECIDE

What should be tried first?

- A

Aggregate first
The report is at store-day grain, so the sort is arranging 30 times more rows than the answer needs. Measured at nearly 7x on this data.
- B

Double the cluster
Buys parallelism for a sort that should not be happening at this size, permanently, and the query stays wrong-shaped.
- C

An index
The sort is of a joined intermediate result rather than a base table, so there is no ordered read for an index to provide.
- D

Reschedule it
Accepts 40 minutes as fixed when the plan says it is not, and moves the problem to whoever owns the new slot.

THE SIGNAL

"The report it feeds is at store-day grain." That sentence says the sort is arranging 7 million rows to answer a question about 237,000, and the fix costs nothing.

Every change in this module **rewrites the query**. Which means every one can change the answer.

The rewrite check

```
1 -- before and after must return the same rows
2 (SELECT * FROM before EXCEPT SELECT * FROM after)
3 UNION ALL
4 (SELECT * FROM after EXCEPT SELECT * FROM before);
5
6 -- must be empty. Module eighteen, doing the work.
7 -- Whole-row comparison, NULL-equal, no per-column handling.
```

3 Both directions. A row whose value changed appears in one diff only if you run one direction.

THE TAKEAWAY

Four lines, run once per optimisation. It is the same check module twenty uses for determinism, pointed at two versions of a query instead of two runs of one.

When the rewrite did change the answer

- ◆ **Sometimes the old one was wrong**, and the rewrite fixed it by accident. This happens more than people admit.
- ◆ **Sometimes the new one is wrong**, usually because a filter moved across a window boundary.
- ◆ **And sometimes both are defensible**, because aggregating earlier changed which population a measure covers.
- ◆ **The diff cannot tell you which**. It tells you a decision is required, which is the whole of its value.

THE TAKEAWAY

A diff that comes back non-empty is not a failed optimisation. It is a question nobody had noticed needed answering, which is usually about grain.

What to record when you optimise

```
1 change:          aggregate to store-day before the ranking
2 why:             the report is at store-day grain
3 rows into sort:  7,069,736 -> 237,312
4 median of 5 runs, both warm: 1.36s -> 0.20s
5 rewrite check:   EXCEPT both ways, empty
6
7 -- five lines. Anybody can reproduce or challenge every one.
```

3 The row count is the claim. It does not vary with load, cache, cluster size or time of day.

5 And the rewrite check is what makes the timing worth reporting at all.

THE TAKEAWAY

Five lines in a commit message or a ticket. The row reduction is the durable claim; the timing is context; the diff is the evidence it is the same query.

Reading a plan, in four steps

01

Find the time

Which single operator holds most of it?
Usually one does, and usually it is a sort.

02

Look at its input

How many rows went in? That is the number to attack, not the operator.

03

Check the estimate

Estimated against actual. A large divergence means the plan was chosen for a different query.

04

Reduce

Can fewer rows reach it? Filter earlier, aggregate earlier, share the sort.

THE TAKEAWAY

Step two is the one people skip. Finding the slow operator feels like finding the problem, and it is only finding where the problem shows up.

Doing less, five ways, with numbers

THE CHANGE	MEASURED HERE
Reduce to the question's grain before the sort	1.36s -> 0.20s (7x)
Share window specifications	7.16s -> 4.69s (1.5x)
Let a filter reach the scan	0.086s -> unmeasurable
Correct statistics	nested loop -> hash join
Materialise a result several things read	N computations -> 1

THE TAKEAWAY

Five changes, and every one gives an operator fewer rows or removes an operator.
None makes an operator faster, because you cannot.

Some queries are **correctly slow**. Reading seven million rows takes as long as it takes.

When slow is correct

- ◆ **The answer genuinely depends on every row**, and no aggregation earlier would preserve it. A true median, for instance.
- ◆ **The output is large**. Producing ten million rows takes as long as producing ten million rows.
- ◆ **Then the remaining levers are physical**, which is partitioning, clustering, more machine, or running it less often.
- ◆ **And running it less often is the one people forget**, because most nightly jobs rebuild things that did not change.

THE TAKEAWAY

The last point is module twenty-two's subject. A correctly slow query that runs hourly and could run daily is a twenty-four-fold saving with no rewriting at all.

Five questions before optimising anything

- ◆ **Is it actually slow, or is it queueing?** A shared warehouse makes everything look slow at the same time.
- ◆ **Does it need to run this often?** The cheapest optimisation is not running, and it needs no rewriting at all.
- ◆ **How many rows does the answer need?** Compare that with the rows entering the expensive operator.
- ◆ **What does the plan say holds the time?** Before forming any theory about what is wrong.
- ◆ **And is there an assertion suite?** Because without one you cannot tell a faster query from a different one.

THE TAKEAWAY

Two of the five are answered without opening the query. A query that is queueing or running hourly when it could run daily does not need optimising at all.

The order to try things in

- ◆ **1. Run it less often**, or not at all. Free, and it is the only change that cannot break the answer.
- ◆ **2. Reduce the rows** reaching the expensive operator. Measured at seven times here, and it costs a subquery.
- ◆ **3. Remove an operator**, by sharing a sort or materialising something computed repeatedly.
- ◆ **4. Fix the statistics**, so the planner stops choosing for a query you are not running.
- ◆ **5. Buy something**. Always available, always works, and it is fifth for a reason.

THE TAKEAWAY

In order of cost to you and risk to the answer. The first two cover most cases, and the fifth is where most conversations start.

A plan is **a description of where the rows went**. It never says what was wrong, only where it showed.

And what to do with a plan you cannot fix

- ◆ **Write down what you tried and what it measured**, because the next person will otherwise try the same four things.
- ◆ **Record the row counts**, so that a change in them later is visible as the cause of a regression.
- ◆ **And state plainly that it is correctly slow**, if it is, which stops the question being reopened every quarter.
- ◆ **A documented dead end is a result.** An undocumented one gets rediscovered at full price.

THE TAKEAWAY

'We tried these four things, here are the numbers, it is scanning everything because the answer needs everything' is a genuinely useful artefact.

What Part 7 has given you so far

- ◆ **Module twenty: seven kinds of assertion**, and one anchor reaching outside the pipeline.
- ◆ **Module twenty-one: read the plan, attack the input**, and measure the folklore before adopting it.
- ◆ **In that order deliberately**. Every optimisation here is a rewrite, and a rewrite needs assertions to be safe.
- ◆ **And module twenty-two is the third question**: not is it right, not is it fast, but is it worth what it costs.

THE TAKEAWAY

Right, fast, affordable. Most teams argue about the second and have no process for the first or the third.

The confusions, settled

BELIEF

COUNT(1) is faster than COUNT(*)

SELECT * is slow

The slow operator is the problem

An index fixes a slow sort

Cost estimates are comparable across queries

A bigger cluster is a fix

THE TRUTH

Identical. Both were unmeasurable on 7 million rows.

Not when the optimiser can prune. It is bad for other reasons.

It is where the problem shows. Its input is the problem.

Only a sort of a base table. Not one of an intermediate result.

They are unitless and only meaningful within one plan.

It is a purchase. It works on every query, including the wrong ones.

THE TAKEAWAY

Rows one and two were measured on the capstone database for this module. The rest follow from what a plan is: a description of where rows went, not of what was wrong.

Five habits

- ◆ **Read the plan before changing anything**, and find the one operator holding the time before forming a theory.
- ◆ **Attack the input, not the operator**. The question is why that many rows arrived, not how to sort them faster.
- ◆ **Compare estimated with actual**, because a large divergence makes every other observation unreliable.
- ◆ **Share window specifications with a named WINDOW clause**, which is measured at one and a half times here.
- ◆ **And measure the folklore before adopting it**, because most of it is a true rule with a false reason.

THE TAKEAWAY

Habit two is the whole module. Every measurement here came from giving an operator fewer rows, and none from making an operator faster.

Module twenty-two closes Part 7 with the thing both modules have been circling: cost, and who pays it.